

TIC-TAC-TOE ARENA

JAVA PROGRAMMING PROJECT REPORT

A modern desktop application featuring an interactive Graphical User Interface (GUI), CardLayout state management, animated turn feedback, and real-time score tracking.

SUBJECT: Java Programming

DEPARTMENT: Artificial Intelligence and Data Science

INSTITUTION: Chaitanya Bharathi Institute of Technology (CBIT)

Submitted By:

Name: V. Praveen Kumar Reddy

Roll No: 1601-25-771-129

Section: AID-2

Academic Year: 2026-2027

1. INTRODUCTION

The **Tic-Tac-Toe Arena Application** is a desktop Graphical User Interface (GUI) program developed in Java using the Swing and AWT frameworks. Unlike traditional command-line applications, this application delivers a rich, modern dark-themed user experience complete with visual landing screens, interactive hover states, dynamic game state handling, and score tracking across multiple rounds.

The project leverages core Object-Oriented Programming (OOP) principles and Event-Driven Architecture to present a seamless sequence between a cover interface ("Enter Arena") and an interactive 3×3 game grid.

2. OBJECTIVES

- To design an appealing graphical desktop application using Java Swing components.
- To implement screen switching using CardLayout without destroying component state.
- To construct an dynamic 3×3 grid system using event handlers to control turns between 'X' and 'O'.
- To evaluate winning patterns and draw conditions algorithmically after every turn.
- To track player metrics continuously, including Player X Wins, Player O Wins, and Draws.
- To provide convenient control options for clearing the grid for new rounds or exiting to the main menu.

3. FEATURES OF THE APPLICATION

- **Cover Screen Landing:** Modern landing page with custom typography, glowing symbol branding, and an direct entry button.
- **Real-Time Score tracking:** Tracks ongoing match wins for Player X, Player O, and total Draws across sessions.
- **Dynamic Turn Visuals:** Dynamic state labels highlighting active player turns with custom brand colors.
- **Interactive Grid Effects:** Custom mouse adapters trigger hover responses over open grid cells.
- **Automated Game Logic:** Instant win/draw condition evaluations along with game grid locking on round completion.

4. TECHNOLOGIES AND TOOLS USED

Technology / Tool	Purpose
Java SE (JDK 25)	Core language runtime and development platform used for project execution.
Java Swing (javax.swing)	GUI toolkit providing lightweight visual elements like JFrame, JPanel, JButton, and JLabel.
Java AWT (java.awt)	Layout managers (CardLayout, BorderLayout, GridBagLayout, GridLayout) and Event Handling.
Visual Studio Code	Primary Integrated Development Environment (IDE) used for writing and debugging code.

5. OBJECT-ORIENTED & GUI CONCEPTS USED

5.1 Inheritance & Layout Architecture

The main window extends JFrame, acting as the top-level window container. Structural layout management is maintained via CardLayout, allowing smooth transitions between the introductory landing screen container and the active game grid panel.

5.2 Encapsulation & State Control

Game state variables including current active player character (turn), round status (over), move counters, and ongoing scores (xWins, oWins, draws) are securely encapsulated inside class scope.

5.3 Event-Driven Programming

User interactivity is managed via standard ActionListener delegates for button clicks and continuous MouseAdapter instances to create mouse-enter and mouse-exit hover highlights.

6. CLASS STRUCTURE & FUNCTIONALITY

Class / Method	Role & Description
TicTacToeScoreApp	Main frame class encapsulating UI elements, state logic, and event listeners.
play(int idx)	Handles player move logic, updates button text/colors, and checks win states.
checkWin()	Iterates over the 8 winning linear index combinations to detect match conditions.
updateScoreText()	Refreshes the ongoing score display label with current win/draw totals.
resetBoard() / resetScores()	Clears the grid for subsequent matches or resets score counters for main menu exit.

7. ALGORITHM & WORKFLOW

1. Start execution via SwingUtilities.invokeLater() inside main().
2. Initialize the JFrame window, setting default dimensions (400×520), dark background color, and card layout container.
3. Render Cover Landing view containing project branding and the **Enter Arena** button.
4. Upon clicking **Enter Arena**, switch card view to active Game Screen.
5. Listen for user grid clicks; if cell is empty and game is active:
    - Set button text to active turn character ('X' or 'O') with matching color styling.
    - Increment total turn moves counter.
    - Execute checkWin() matrix scanning. If win detected, update score counters and lock grid.
    - If move counter equals 9 without a win, set state to Draw and update draw metric.
    - Otherwise, toggle active player turn variable.
6. Allow user to click **NEXT ROUND** to reset board cells or **MAIN MENU** to clear scores and return to cover screen.

8. UI DESIGN & COVER SCREEN ARCHITECTURE

The application interface features a dark design, implemented using custom RGB colors.

✕ ○

Tic-Tac-Toe

Experience classic strategy with smart visual indicators & real-time scores

🎮 **Enter Arena**

10. TESTING & VERIFICATION

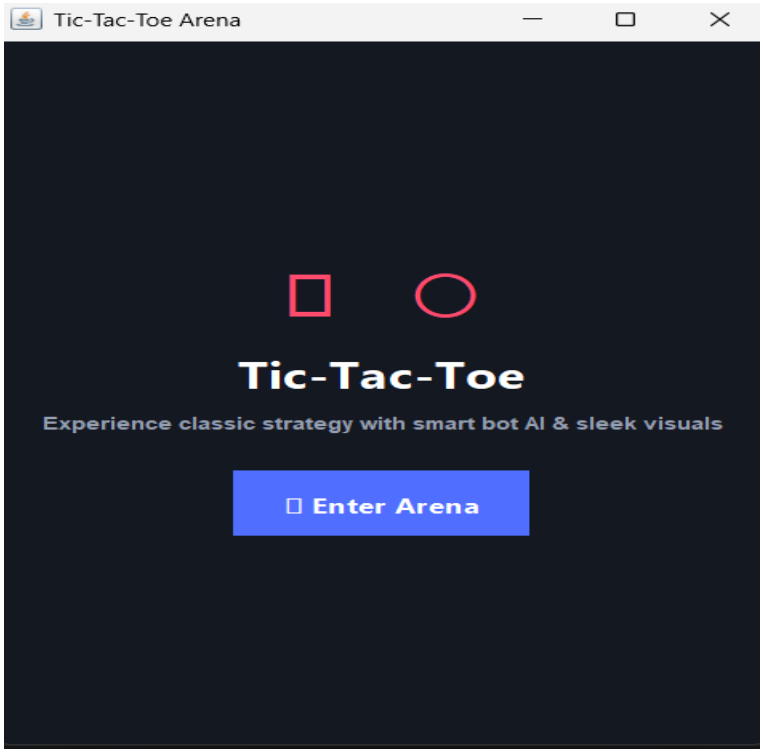
Test Case	Input / Action	Expected Result	Status
TC-01	Click "Enter Arena"	Switches transition from Cover card to interactive Game grid.	**PASS**
TC-02	Click empty cell	Displays 'X' or 'O' in custom color & toggles next player status.	**PASS**
TC-03	Complete 3-in-a-row	Triggers win message, updates score header, and locks grid cells.	**PASS**
TC-04	Fill grid without 3-in-a-row	Triggers "Draw" message and increments Draw metric counter.	**PASS**
TC-05	Click "Next Round"	Resets board buttons without resetting session score accumulators.	**PASS**

11. ADVANTAGES

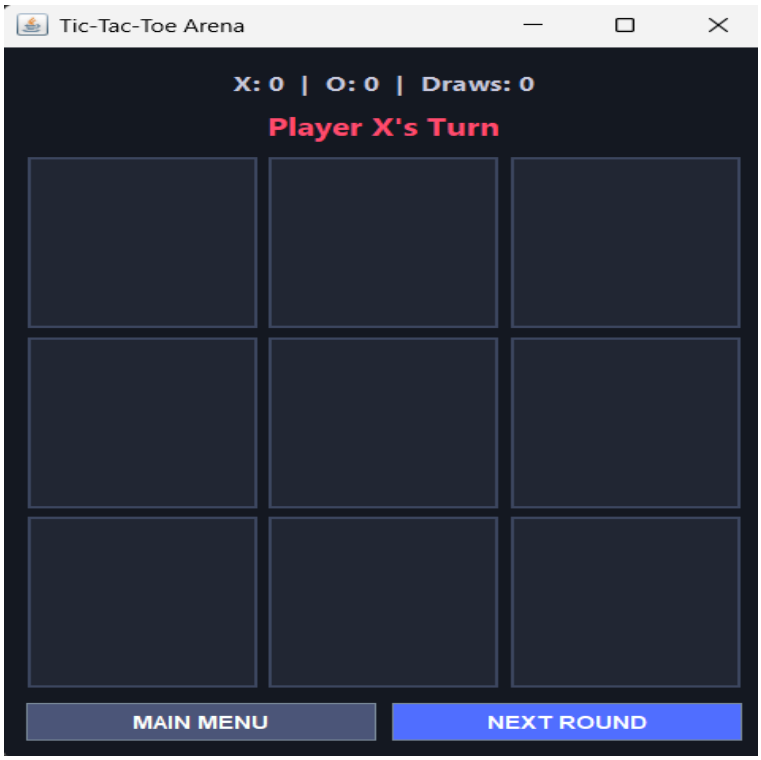
- **Intuitive Modern UI:** Dark-mode visual theme built for high contrast and modern aesthetics.
- **Persistent Round Tracking:** Tracks wins and draws across multiple gaming rounds continuously.
- **Smooth Page Navigation:** Employs CardLayout transitions without requiring extra frames or windows.
- **Extensible Architecture:** Compact code structure allows simple integration of Single-Player AI algorithms.

12. OUTPUT SCREENSHOTS

12.1 INTERFACE OF PAGE:



12.2 GAME SCREENSHOT



12.3 FINAL RESULT SCREENSHOT



13. LIMITATIONS & FUTURE ENHANCEMENTS

Limitations: Current execution supports two-player local desktop gameplay only without network multiplayer or file storage for long-term score retention.

Future Enhancements:

- Integration of an AI opponent using the Minimax algorithm for single-player mode.
- File handling or database integration to save high scores permanently.
- Adding sound effects and custom animations for winning combinations.

14. CONCLUSION

The **Tic-Tac-Toe Arena Application** was successfully implemented using Java Swing and AWT. The project demonstrates core Java principles including Event-Driven UI design, custom color themes, component state retention, and array algorithm validation, delivering a complete project package.

15. REFERENCES

- Oracle Java Documentation - javax.swing & java.awt APIs
- Java Swing GUI Programming Tutorials & Documentation
- Class notes and laboratory material
- Java Development Kit (JDK) documentation